Chloride Exide KE — Battery Finder: Technical Documentation

Version: 2.0 | **Date:** 2026-08-30 | **Stack:** FastAPI · PostgreSQL · React 19 · TypeScript · Tailwind CSS v4

Table of Contents

1. System Overview
2. High-Level System Diagram
3. Backend Architecture
4. Database Schema
5. The Matching Engine — Deep Dive
6. Matching Mode 1: Select Mode
7. Matching Mode 2: Type Mode
8. Authentication & Security
9. Admin Dashboard API
10. Frontend Architecture
11. Data Pipelines
12. Rate Limiting & Observability
13. Automated Test Suite
14. Deployment & Environment

1. System Overview

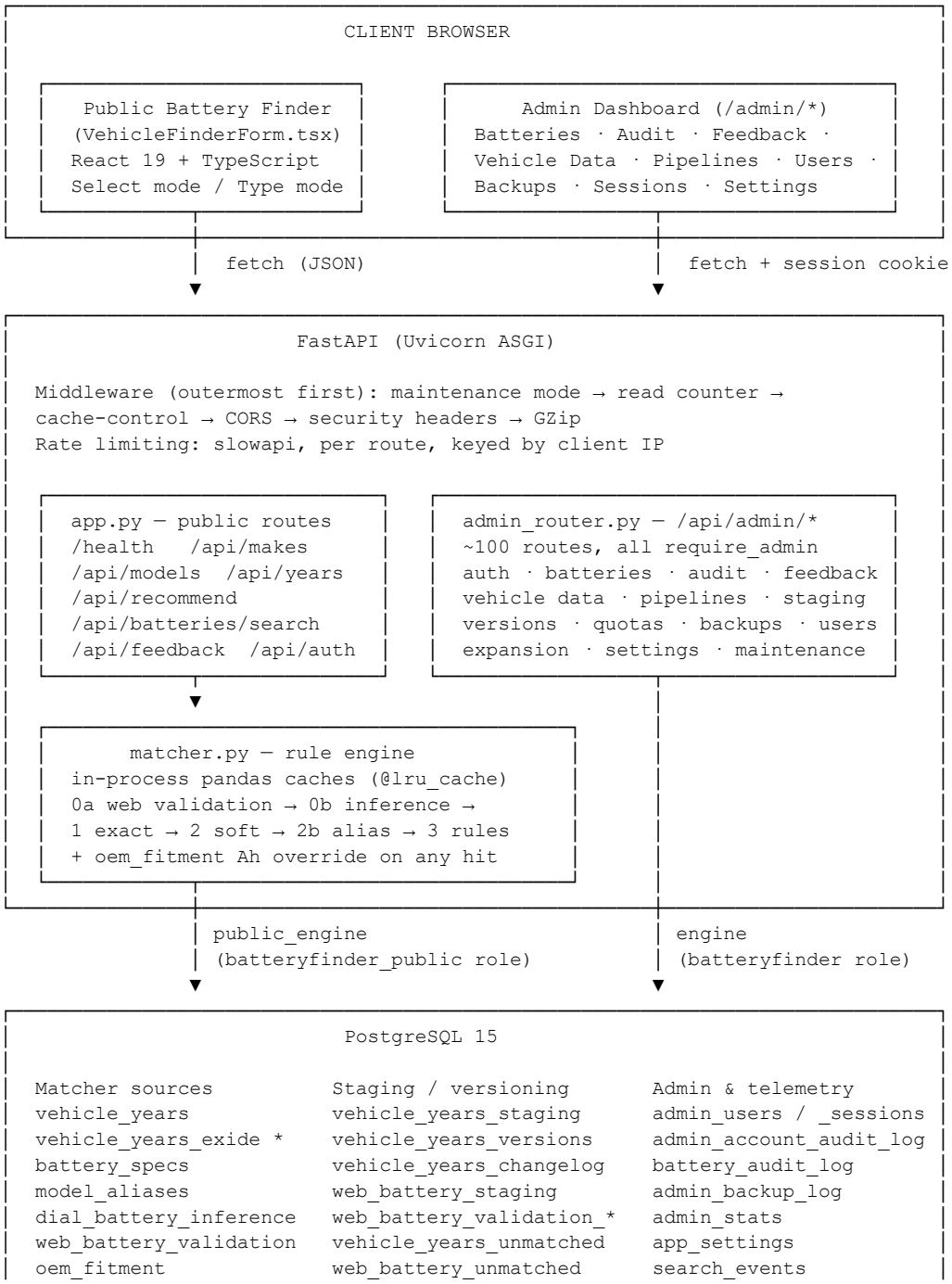
The Battery Finder is a production web application that recommends the correct car battery for Kenyan vehicles. Users enter their vehicle details — make, model, year, fuel type, and engine CC — and the system returns a primary battery recommendation with same-technology alternatives. The application is geo-restricted to Kenya, Uganda and Tanzania and is built to serve the Chloride Exide KE brand.

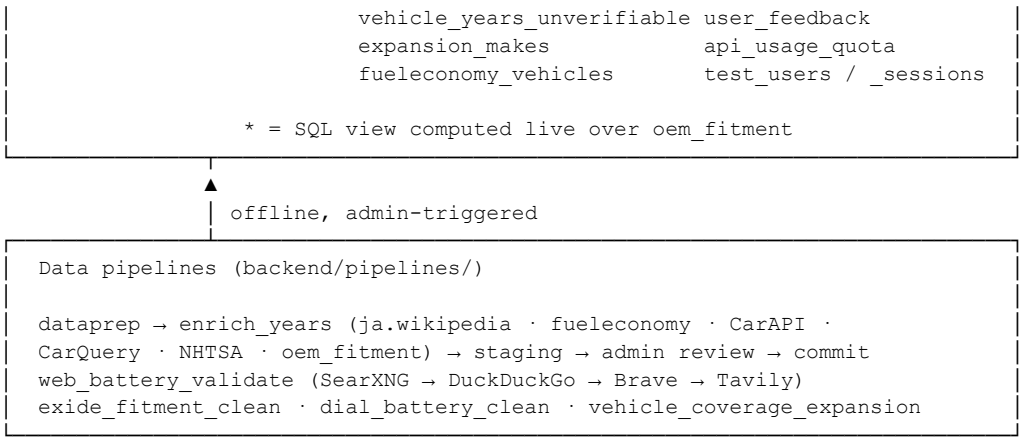
Core technologies:

Layer	Technology
Backend API	Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic, slowapi
Matching	pandas DataFrames, rapidfuzz (fuzzy text matching), rule engine
Database	PostgreSQL 15 — two roles, two pools (see 8.5); pool size env-tunable
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, Bun
Auth	bcrypt (cost 12), HttpOnly cookies (admin), Bearer tokens (tests)
Serving	Uvicorn (ASGI); FastAPI serves the built React SPA
Testing	pytest (backend, real Postgres), Vitest (frontend), Jenkins CI

What changed since version 1.0. The recommendation engine gained two new data sources — an internet-sourced fitment table (`web_battery_validation`) and Exide's own OEM fitment guide (`oem_fitment`) — so the matching cascade is now five layers deep with a separate Ah override sitting on top of it (section 5). The database grew from roughly ten tables to twenty-eight plus a view, largely for the staging and versioning workflow that now fronts every bulk data change. The public request path moved onto its own least-privilege Postgres role, and admin accounts became fully self-service (bootstrap, forgot-password, user management) rather than requiring a CLI script on the server.

2. High-Level System Diagram





3. Backend Architecture

3.1 Entry Point: `app.py`

`app.py` is the FastAPI application root. On startup (via the `lifespan` async context manager) it:

1. Verifies PostgreSQL connectivity with a `SELECT 1`.
2. Warms all five matcher LRU caches (`_vehicles`, `_batteries`, `_model_aliases`, `_inference`, `_fitment`) so the first user request is not cold.
3. Starts a background `threading.Timer` that flushes the in-memory read counter to `admin_stats` every 60 seconds.
4. Schedules a midnight `threading.Timer` that resets the daily read counter to zero.
5. Loads the persisted maintenance-mode flag from the `app_settings` table into the per-process cache.

Public API routes registered in `app.py`:

Method	Path	Rate Limit	Description
GET	`/health`	60/min	Unauthenticated liveness/readiness probe for the load balancer
GET	`/api/makes`	20/min	List all car makes
GET	`/api/models/{make}`	20/min	List models for a given make
GET	`/api/years`	20/min	List years for a make + model
POST	`/api/recommend`	10/min	Get a battery recommendation
GET	`/api/batteries/search`	20/min	Find batteries within ±15% of a target Ah and/or CCA
POST	`/api/feedback`	5/min	Submit feedback on a recommendation
POST	`/api/auth/register`	5/min	Register a test user
POST	`/api/auth/login`	10/min	Log in as a test user
POST	`/api/auth/logout`	—	Log out a test user
GET	`/api/auth/me`	—	Current test user info

`POST /api/recommend` request body:

```
{
  "make":       "TOYOTA",
  "model":      "Corolla",
  "year":       2015,
  "engine_cc":  1500,
  "fuel_type":  "Petrol",
  "stop_start": true,
  "input_type": "select"
}
```

`year` is validated to 1960–2030, `engine_cc` to 500–8000, and `input_type` to `select` or `type`.

`POST /api/recommend` response:

```
{
  "make":           "TOYOTA",
  "model":          "COROLLA",
  "year":           2015,
  "engine_cc":      1500,
  "fuel_type":      "PETROL",
  "battery_region": "NS",
  "recommended_ah": 55,
  "preferred_tech": "EFB",
  "confidence":     100,
  "match_type":     "Exact Match",
  "matched_make":   "TOYOTA",
  "matched_model":  "COROLLA AXIO G",
  "matched_year":   2015,
  "ah_source":      "oem_fitment",
  "primary":        { "battery_code": "...", "ah": 55, "cca_en": 480, ... },
  "alternatives":   [ ... ]
}
```

`ah_source` is new in version 2.0 and records which path actually supplied the Ah figure: `oem_fitment`, `battery_code`, or `cc_bracket_estimate`. See section 5.

3.2 Middleware Stack

Middleware is registered so that maintenance mode runs outermost and GZip innermost:

Middleware	Purpose
`maintenance_mode_middleware`	Serves `static/maintenance.html` with HTTP 503 for every path except `/api/admin/*`, `/admin/*`, `/static/*`, `/assets/*` and `/health`
`count_reads_middleware`	Increments the in-memory read counter for `POST /api/recommend` only
`cache_control_middleware`	Adds `Cache-Control: public, max-age=31536000, immutable` to `/assets/*` (Vite hashes those filenames)
`CORSMiddleware`	Allowlist from `ALLOWED_ORIGINS`; empty by default because the SPA and API are served same-origin
`security_headers`	`X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, `Content-Security-Policy`, plus `Strict-Transport-Security` when cookies are in secure mode
`GZipMiddleware`	Compresses responses larger than 500 bytes

Two exception handlers are registered: `RequestValidationError` (flattens Pydantic errors into one readable 422 message) and `RateLimitExceeded` (returns a clean 429 with `retry_after` instead of an unhandled 500).

3.3 Read Counter Middleware

Every `POST /api/recommend` request increments `_read_counter_pending` (an in-memory integer guarded by a `threading.Lock`). One timer flushes the accumulated delta to `admin_stats` every 60 seconds in a single write; a second zeroes it at local midnight. This keeps the dashboard's "reads today" figure accurate without adding a database write to the request path.

Separately, each recommendation spawns a short-lived daemon thread that writes a `search_events` row (vehicle asked for, battery returned, match type, confidence, truncated IP, input mode). Failures there are logged at debug level and never surface to the caller.

3.4 Static File Serving & Maintenance Mode

`app.py` mounts:

- `/assets/*` — Vite-built JS/CSS chunks from `frontend/dist/assets/`
- `/static/*` — raw static files (maintenance page, logos)
- `/{full_path}` catch-all — returns React `index.html` for all non-API paths (SPA client-side routing)

The catch-all resolves the requested path with `os.path.normpath()` and serves it only if the result still sits inside the frontend root — the root is compared with a trailing separator so a sibling directory such as `dist-evil` cannot pass as `dist`. Anything that escapes falls through to `index.html`. This closes a path-traversal hole that previously let URL-encoded `../` sequences read arbitrary backend files, including `.env`.

Maintenance mode is **not** an environment variable. It is a row in `app_settings` (`setting_key = 'maintenance_mode'`), toggled from the admin Settings page and cached per process for 5 seconds, so every worker and every instance converges within 5 seconds of a toggle without needing Redis.

4. Database Schema

All migrations are managed by Alembic (in `alembic/versions/`). Run `alembic upgrade head` from `backend/` after pulling. The schema is now 28 tables plus one SQL view, grouped below by role.

4.1 Runtime Tables (read by the matcher)

These seven relations are the only ones the recommendation path touches. All are loaded into memory at startup and re-read only on an explicit cache refresh.

`vehicle_years` — Primary vehicle lookup table. One row per make/model/year combination.

Column	Type	Constraints	Description
id	Integer	PK, autoincrement	
make	String(100)	NOT NULL	Vehicle make, e.g. "TOYOTA"
model	String(200)	NOT NULL	Model/trim as it appears in the source data
year	Integer	NOT NULL	Model year
engine_capacity	Integer	nullable	Engine displacement in CC
fuel	String(50)	nullable	Raw fuel string,

			normalised at load time
source	String(50)	nullable	Which enrichment source produced the row
model_number	String(100)	nullable	CRSP chassis/type-approval code, e.g. "3BA-VJA300W-GNUZZ"

Unique constraint: `(make, model, year)`. Indexes on `make` and `(make, model)`. `source` and `model_number` were added so every row carries its provenance — historically 98% of rows had no source recorded, which made bad year data impossible to attribute.

`vehicle_years_exide` — **A SQL view, not a table.** It expands `oem_fitment`'s year ranges into one row per make/model/year, strips trailing chassis codes from the model string, and caps every range at the current year. `matcher._vehicles()` reads `vehicle_years UNION ALL vehicle_years_exide`, so Exide's fitment-guide vehicles are searchable without ever being written into `vehicle_years`. Because it is a view, it updates the moment `oem_fitment` changes — there is no separate refresh step.

`battery_specs` — Battery catalogue. One row per battery SKU.

Column	Type	Constraints	Description
id	Integer	PK	
battery_code	String(100)	NOT NULL, unique	SKU code
model_key	String(100)	NOT NULL	Standard prefix — "DIN…" means DIN, anything else NS
ah	Float	nullable	Ampere-hours
cca_en	Float	nullable	Cold cranking amps (EN rating)
technology	String(100)	nullable	EFB / AGM / MF / SB (Flooded)
origin	String(50)	nullable	`local` or `imported`
brand	String(100)	nullable	e.g. Powerlast, Sebang
deleted_at	DateTime(tz)	nullable	Soft delete — always filter `IS NULL`

Indexes on `model_key` and `ah`.

`model_aliases` — International → JDM model name mappings, with a `priority` column (lower wins) used to break ties when one international name maps to several JDM nameplates.

`dial_battery_inference` — Real-world call-centre order data: which battery was actually sold, per vehicle type and model, with a `frequency` count and `last_sale_date` tiebreaker.

`web_battery_validation` — Internet-sourced fitment, one row per `(make_n, model_n, year)` with `battery_code`, `ah`, `technology`, `source` and `validated_at`. Populated offline by `pipelines/web_battery_validate.py`. Unique on `(make_n, model_n, year)`; indexed on `make_n` and `(make_n, model_n)`.

`oem_fitment` — Exide's official global fitment guide, kept as year ranges rather than exploded rows.

Column	Type	Description

vehicle_category	String(20)	`passenger` or `commercial`
make / model	String	Model is the raw trim string, e.g. "OUTLANDER III 2.2 DI-D"
year_from / year_to	Integer	`year_to` NULL means open-ended
ah	Float	Fitment Ah — this is what overrides the matcher's own figure
technology	String(50)	Cleaned value; `raw_technology` keeps the original
dimensions	String(50)	Raw, e.g. "242*175*190"
cylinder_capacity_cc	Integer	Parsed from strings like "1368cm"
source_row	Integer	Original Excel row, for debugging

A single make/model/year-range legitimately has **several** rows — Exide lists multiple battery tiers for the same trim (a base 50Ah/MF option alongside 55Ah/EFB and 60Ah/EFB upgrades). That is not duplicate data, and the override logic in section 5 picks among the tiers deliberately.

4.2 Pipeline, Staging & Versioning Tables

Every bulk data change now goes through staging and is revertible.

Table	Purpose
`vehicles_raw`	Output of Stage 1 (`dataprep.py`) — deduplicated make/model/CC/fuel rows from the CRSP Excel
`vehicle_years_staging`	Stage 2 output awaiting review. Carries `is_valid`, `invalid_reason`, `conflict` and `conflict_detail` (JSON showing current vs staged)
`vehicle_years_versions`	One row per commit: counts of added/updated/removed/conflicting rows, who initiated and who committed
`vehicle_years_changelog`	Row-level diffs per version (`added` / `updated` / `removed` with old and new JSON values) — this is what makes a revert possible
`vehicle_years_unmatched`	Make/model pairs no source could resolve in a given run
`vehicle_years_unverifiable`	Rows quarantined out of `vehicle_years` because their years could not be verified; keeps `base_model`, `reason` and a `batch_id` so a run can be restored
`web_battery_staging`	Same review workflow for the web-validation pipeline
`web_battery_validation_versions` / `_changelog`	Version history and row-level diffs for web-validation commits
`web_battery_unmatched`	Web-validation rows rejected as implausible, with both the web's Ah and the rules' expected Ah for comparison
`pipeline_run_log`	Per-run bookkeeping
`expansion_makes`	Target makes for the coverage-expansion pipeline, with per-source slugs and a `year_tier` (default / commercial / chinese)
`fueleconomy_vehicles`	Cached fueleconomy.gov bulk CSV, loaded into the DB rather than re-parsed per run
`enrichment_cache_meta`	Freshness metadata per cached dataset — drives the "cache is stale" prompt in the admin UI
`api_usage_quota`	Per-provider call accounting. `quota_limit = -1` means unlimited; a NULL `period_end` marks a

	prepaid one-time balance that never auto-resets

4.3 Admin & Audit Tables

Table	Purpose
`admin_users`	bcrypt credentials plus `failed_login_attempts`, `locked_until`, `email`, and a SHA-256 `reset_token_hash` with its expiry
`admin_sessions`	One row per active admin login, 8-hour TTL, 64-char hex token
`admin_account_audit_log`	Bootstrap / create / delete / reset-link / password-reset events. Actor and target usernames are stored as snapshots, not just foreign keys, so entries stay readable after an account is deleted
`battery_audit_log`	Immutable change history for `battery_specs`, old and new values as JSON
`admin_backup_log`	One row per backup attempt: filename, size, status, error detail, who triggered it
`admin_stats`	Key-value counters; currently `db_reads`
`app_settings`	Key-value application settings; currently `maintenance_mode`
`search_events`	One row per recommendation: vehicle asked for, battery returned, match type, confidence, whether it matched, truncated IP, and input mode

4.4 User Tables

Table	Purpose
`user_feedback`	Satisfaction (1–5), accuracy rating, which parts were wrong (JSON), comments, plus the match metadata the recommendation returned
`test_users`	Registered test-user accounts (bcrypt)
`test_user_sessions`	Bearer-token sessions, 24-hour TTL, with `last_seen_at`

5. The Matching Engine — Deep Dive

`matcher.py` is the recommendation core. It runs entirely in-process against pandas DataFrames held in LRU caches. No database round-trip happens during a recommendation request.

5.1 Data Loading & Caching

At startup, `app.py` warms five caches; two more load lazily on first use:

```
matcher._vehicles()       → vehicle_years UNION ALL vehicle_years_exide
matcher._batteries()      → battery_specs WHERE deleted_at IS NULL
matcher._model_aliases()  → model_aliases
matcher._inference()      → dial_battery_inference
matcher._fitment()        → oem_fitment
matcher._web_validation() → web_battery_validation      (lazy)
matcher._make_index(make) → per-make row slice + model → years map (lazy, 128 makes)
```

Each is decorated with `@lru_cache`. All read through `public_engine` — the least-privilege role (section 8.5). The caches are cleared and immediately re-warmed whenever an admin edits battery specs or triggers `POST /api/admin/cache/refresh`.

`_make_index()` deserves a note: it precomputes, once per make, the make-scoped row slice and a `model_n → sorted years` map. Before it existed, both `recommend()` and the per-candidate year filtering inside fuzzy matching re-scanned the full vehicle table on every call.

Normalised columns added at load time:

- `make_n` — uppercase + strip + collapse whitespace
- `model_n` — result of `_norm_model_key()`: uppercase, hyphens → space, and a space inserted at every letter↔digit boundary, so `CX5`, `CX 5` and `CX-5` all resolve to the same key
- `fuel_n` — mapped through `_FUEL_MAP`, which folds vendor spellings and source-data typos ("DEISEL" → DIESEL, "ELECCTRIC" → EV, "PLUG-IN HYBRID" → HYBRID)
- `vtype_n`, `vmodel_n`, `bcode_n` — the same normalisation applied to the inference table

5.2 The `recommend()` Function — Layer Order

`recommend(make, model, year, engine_cc, fuel_type, stop_start) → dict`

Pre-processing:

1. Normalise `make_n` = `make.upper().strip()`
2. Normalise `model_n` = `_norm_model_key(model)`
3. Set defaults: `resolved_cc = engine_cc or 1500`, `resolved_fuel = _norm_fuel(fuel_type)`
4. Set `confidence = 40`, `match_type = "Estimated (Rule-Based)"` as the floor

The engine then tries layers in order and **returns at the first successful match**:

Layer	Source	Confidence	`match_type`
0a	`web_battery_validation` — internet-sourced, year-specific	75–90	Web-Validated Match
0b	`dial_battery_inference` — real call-centre sales	80–95	Real-World Data Match
1	`vehicle_years` — fuzzy model ≥ 90 + exact year	100	Exact Match
2	`vehicle_years` — fuzzy model + year within ±3	60–89	Close Match
2b	`model_aliases` — international → JDM, then re-run 1/2	60–100	Exact / Close Match
3	Rules — engine CC + fuel → Ah bracket	40	Estimated (Rule-Based)

Layers 0a and 0b take priority over 1–3 because they reflect actual fitment outcomes rather than a rule-of-thumb estimate. They only fire when a matching row exists in their source table, so most searches still fall through to layers 1–3.

On top of all of this sits the OEM fitment override (described after Layer 3). It is not a competing layer: it never changes which layer matched or the resulting confidence, only the Ah figure that layer produces.

Layer 0a: Web-Validated Match

What it does: Looks up `web_battery_validation` for this exact `(make, model, year)`. The table is populated offline by `pipelines/web_battery_validate.py`, which searches the open web for the battery commonly fitted to each vehicle.

How it works:

1. Filter to rows where `make_n` matches, then to rows where `year` matches exactly. A miss on either ends the layer — unlike inference, this source is year-specific and is never stretched across years.
2. Exact `model_n` hit → confidence 90. Otherwise fuzzy match with `token_set_ratio ≥ 85` → confidence 75.
3. Engine CC and fuel are then resolved from `vehicle_years` for reporting only — exact year first, else the nearest row within ±3 years.
4. The stored `battery_code` is looked up in `battery_specs` to yield `(target_ah, technology)`.
5. Technology is decided the same way as Layer 0b (below): a luxury make past its AGM year always gets AGM; `stop_start=False` recomputes from rules; AGM/EFB in the source is honoured; MF is recomputed for stop-start vehicles.

Why it outranks inference: web validation reflects current fitment standards, while inference reflects historical sales — including sales that were themselves wrong recommendations.

Layer 0b: Inference Match (Real-World Order Data)

What it does: Looks up `dial_battery_inference`. This table contains actual Dial-A-Battery call-centre sales data — if a battery was sold for a Toyota Corolla 100 times, that is strong evidence.

How it works:

1. Filter inference rows where `vtype_n == make_n`.
2. Exact `model_n` hit → confidence 95; else fuzzy `token_set_ratio ≥ 85` → confidence 80.
3. Among all rows for the matched (make, model), pick the battery with the **highest frequency**, tie-broken by the most recent `last_sale_date`.
4. Look up that battery code in `battery_specs` (exact → fuzzy → legacy parser) to get `(target_ah, technology)`.
5. **Technology override logic:**

- Luxury make at or past its AGM adoption year → AGM regardless of anything else.
- `stop_start=False` → recompute from rules (yields MF).
- Source returned AGM or EFB → honour it; a call centre upgrading the chemistry was a deliberate decision.
- Source returned MF and `stop_start=True` → recompute via `_preferred_tech()`. MF codes in the inference table often reflect non-i-stop sales or outright wrong recommendations, so they are not copied blindly onto stop-start vehicles.

1. Call `_select_batteries(target_ah, pref_tech, region)` and return.

Return confidence: 80 (fuzzy model match) or 95 (exact model match). `match_type = "Real-World Data Match"`. The frontend `AccuracyBar` awards full scores for both Car Found and Data Quality segments when this layer fires.

Layer 1: Exact Match

Reached only if neither Layer 0 source hit.

Step 1 — Make resolution: Filter the vehicle table to rows where `make_n` matches. If no rows are found, fuzzy-match the make against all known makes with `fuzz.WRatio ≥ 75`.

Step 2 — Model fuzzy match (`_fuzzy_model_score_for_year`): This function is the core of input-to-database matching. An exact `model_n` hit short-circuits it entirely with score 100 — this matters because a longer trim variant can otherwise tie at 100 and win. Otherwise three scorers run in order:

```
1. fuzz.partial_ratio    (threshold ≥ 80, only if len(query) ≥ 4)
   — catches prefix inputs: "COROLLA" matches "COROLLA AXIO G"

2. fuzz.token_set_ratio  (threshold ≥ 72)
   — catches variant-token inputs: "MARCH ACTIVE" matches "MARCH G"
   — ignores word order and extra tokens

3. fuzz.WRatio           (threshold ≥ 60)
   — general-purpose weighted ratio fallback
```

For each scorer, rather than taking the top-scoring candidate, it collects **all** candidates above the threshold and picks the best with a multi-key sort:

```
Priority key: (starts_with_query, is_exact_year, -year_delta, score, model_n)
```

`starts_with_query` is 1 when the candidate model name starts with the user's input. This stops "FIT" from being hijacked by a longer "FIT SPORT" when "FIT SHUTTLE" — whose years actually cover the requested year — also scores 100.

Step 3a — Exact year match: Filter the matched model's rows for `year == year`. If found:

- `confidence = 100` if model_score ≥ 90, else `int(model_score)`
- `match_type = "Exact Match"` if model_score ≥ 90, else `"Close Match"`
- `resolved_cc` and `resolved_fuel` are taken from this row — on a dataset match the dataset is authoritative.
- AccuracyBar: Car Found fills to 50/50; Year fills to 25/25 (zero delta).

Layer 2: Soft Year Match

Reached if no exact year row exists for the matched model.

Step 3b — ±3 year window: Filter the matched model's rows to `year ∈ [year-3, year+3]` and pick the closest year.

```
year_delta  = |nearest["year"] - requested_year|
confidence  = max(60, int(model_score) - year_delta * 3)
match_type  = "Close Match"
```

Example: model score 85, year delta 2 → `confidence = max(60, 85 - 6) = 79`.

AccuracyBar Year segment: delta 0 → 25 pts, delta 1 → 20, delta 2 → 13, delta 3 → 8. Car Found fills proportionally to the model score (20–40 out of 50 pts in close-match range).

Layer 2b: Alias Resolution (runs after Layer 2 fails)

If nothing matched at all (`matched_year is None`) and `enable_aliases=True`, the engine tries international → JDM model aliases:

1. Filter `model_aliases` to the current make.
2. Fuzzy-match `model_n` against `intl_model_n` values with `fuzz.WRatio ≥ 75`.
3. For each matching alias, resolve its `jdm_model_n` values against the vehicle dataset, again at `WRatio ≥ 75`.
4. From all resolved candidates, pick the best by: exact year first, then smallest year delta, then lowest alias `priority`, then highest JDM similarity.
5. Re-run Steps 3a/3b with the resolved JDM model name.

This is how a user typing "COROLLA" reaches "COROLLA AXIO G" or "COROLLA FIELDER" — the alias table maps one international name to several JDM trims, and the engine picks the one whose year range best covers the requested year.

Layer 3: Rule-Based Fallback

Reached if every matching layer failed; confidence stays at 40. AccuracyBar: Car Found = 0, Data Quality = 4 (minimum), Engine Info proportional to whether the CC was non-default. An amber warning banner is shown above the result card when this layer fires (`matched_make` is `null` in the response).

`_target_ah(engine_cc, fuel, year)` — Ah bracket lookup table:

Max CC (inclusive)	Petrol Ah	Diesel Ah	Hybrid Ah
1000	35	40	35
1600	45	50	45
2000	55	65	55
2500	70	80	65
3000	80	90	80
4000	90	100	90
9999+	100	120	100

EVs bypass the table entirely and take a 45 Ah base: the 12V auxiliary battery in an electric vehicle is not correlated with traction-motor size.

Year modifier (only applied in rule-based mode):

Year range	Modifier	Rationale
≥ 2018	+10 Ah	Modern electronics era
2012–2017	+5 Ah	Early ADAS era
2000–2011	0	
< 2000	-5 Ah	Pre-OBD2, minimal electrical load

A guard also rewrites `HYBRID` to `PETROL` when the matched year is before 2000, since pre-2000 hybrid labels in the source data are always errors.

`_preferred_tech(engine_cc, fuel, year, make, stop_start)` — Technology selection, first match wins:

Condition	Technology
EV fuel type	MF
Luxury/premium make **at or past its AGM adoption year**	AGM
Hybrid fuel type	EFB
`stop_start=True` AND (Diesel OR engine_cc ≥ 2500)	AGM
`stop_start=True` (standard petrol)	EFB
`stop_start=False`	MF

The luxury-AGM rule is year-gated, which is new in version 2.0. AGM did not become an OEM fitment on premium cars until stop-start and brake-energy regeneration arrived, so recommending AGM for a 2001 320i both overcharges the customer and misstates the fitment. A per-make adoption table is used rather than one industry-wide date — BMW was fitting AGM in 2007, Maserati not until 2013:

Make	AGM from	Make	AGM from
BMW	2007	Jaguar	2011
Mercedes / Mercedes-Benz	2008	Land Rover	2011
Audi	2010	Bentley	2012
Porsche	2010	Lexus	2012
Volvo	2010	Infiniti	2013
Cadillac	2010	Maserati	2013
Rolls-Royce	2010	*(any other premium make)*	2010
Ferrari	2010		

These are brand-level adoption dates, not per-model fitment facts; they are deliberately set at the point the technology became normal for the brand rather than its first appearance anywhere in the range. Where Exide's own fitment guide covers a vehicle, that data wins — see the override below.

Battery region detection (`_make_region`):

Region	Makes
NS	Toyota, Honda, Nissan, Mazda, Subaru, Isuzu, Hino, Lexus, Daihatsu, Suzuki, Mitsubishi (incl. Fuso), UD, Hyundai, Kia, and the Chinese makes (Chery, Dongfeng, FAW, Foton, Great Wall, Haval, SAIC)
DIN	Everything else — European, American and Indian makes (Audi, BMW, Mercedes, VW, Ford, Chevrolet, Peugeot, Renault, Fiat, Tata, Scania, MAN, …)

This determines whether the `battery_specs` pool is filtered to NS-standard or DIN-standard batteries, detected from the `Model Key` prefix: "DIN…" → DIN, everything else → NS. The region always comes from the vehicle's own make, never from a scraped keyword — trusting a scraped "NS"/"DIN" string previously put DIN-coded batteries on 490 NS-region vehicles.

OEM Fitment Override

Once any layer above has resolved a `(make, model, year)`, `oem_fitment` is consulted for that same vehicle. If a row covers it, **its Ah replaces whatever the layer would otherwise have used** — including a confirmed battery-code Ah from Layer 0a or 0b — because this is Exide's own spec sheet. The response's `ah_source` field records which path actually supplied the figure: `oem_fitment`, `battery_code`, or `cc_bracket_estimate`.

Matching is exact model first, then fuzzy `token_set_ratio ≥ 85`, always constrained to rows whose `[year_from, year_to]` covers the year (a NULL `year_to` is open-ended). Rows with a NULL Ah are skipped — a technology-only hint is not enough to drive battery selection.

Two details matter:

- **Ties are pooled, not picked arbitrarily.** A bare query like "OUTLANDER" is a subset match scoring 100 against many different Exide trims at once. Rather than take whichever one the fuzzy matcher returns first, every model tied at the best score is collected, so the choice is made deliberately in the next step instead of landing on an unrepresentative trim such as the hybrid's 45 Ah.
- **Among tied candidates, the tier whose technology matches the already-decided `preferred_tech` wins**, falling back to highest Ah only when none match. This keeps the Ah figure paired with a chemistry Exide actually offered it at.

`oem_fitment`'s technology column is deliberately NOT applied to `preferred_tech`. Many of its values are Exide product-tier names (PREMIUM, CLASSIC, START PRO) rather than chemistries, and even where the value is a real chemistry, technology preference must keep following this application's own luxury-make and stop-start rules, which know about local stock and brand-tier policy in a way a global guide does not.

Battery Selection (`_select_batteries`)

Once `target_ah`, `preferred_tech` and `region` are known:

1. Filter `battery_specs` to the correct region.
2. Filter to rows where `Ah == target_ah`.
3. Sort by technology preference order: `EFB > AGM > MF > SB (Flooded)`.
4. Within the same technology, sort imported before local.
5. If `preferred_tech` is stocked at `target_ah` → that becomes the primary.
6. If not → take the best available technology at the same Ah.
7. If nothing exists at `target_ah` → take the nearest Ah in any technology.
8. **Alternatives:** same Ah, same CCA and same technology as the *actual* primary — not the target — capped at 6, imported first. Using the primary's real Ah matters for step 7, where the primary may have stepped to a different Ah than requested.

6. Matching Mode 1: Select Mode

Select mode is the default user experience. The form uses three cascading dropdowns.

Interaction flow:

```
User selects Make (dropdown)
  → GET /api/makes (loaded on page mount, cached in React state)
```

```
  → handleMakeChange() fires GET /api/models/:make
  → models dropdown is populated

User selects Model
  → handleModelChange() fires GET /api/years?make=&model=
  → years dropdown is populated

User selects Year
  → form is ready; user may also choose Fuel Type and Engine CC (optional here)

User clicks "Find Battery"
  → POST /api/recommend with { make, model, year, fuel_type?, engine_cc?,
                               stop_start, input_type: "select" }
```

Why select mode can still use fuzzy matching: the request carries the model name exactly as it appears in the database, since the dropdown was populated from `GET /api/models`. The matcher still normalises it through `_norm_model_key()` and tries an exact match against `model_n` first; when that key is found it bypasses fuzzy matching entirely with score 100. Fuzzy matching only comes into play when the model name in `vehicle_years` differs from the one in a Layer 0 source table.

Confidence in select mode:

- A model chosen from the dropdown matches a `model_n` key exactly → model_score = 100.
- Year present in the database → confidence 100, "Exact Match".
- Year absent but within ±3 → confidence 60–97, "Close Match".
- No year within the window → rule-based fallback, confidence 40.

A Layer 0a or 0b hit can pre-empt all of the above and return 75–95 instead, since those layers are consulted before the vehicle table.

7. Matching Mode 2: Type Mode

Type mode allows free-text entry. This is where the full fuzzy matching pipeline matters.

Interaction flow:

```
User types make, model, year in free-text inputs
User must also provide Fuel Type and Engine CC (required in type mode)

User clicks "Find Battery"
  → POST /api/recommend with all fields, input_type: "type"
```

Estimated confidence indicator (frontend only): Before submitting, the form shows a live estimate to encourage users to fill in every field:

Fields filled	Shown confidence	Label
Make only	40%	"Low — only make entered"
Make + Model	55%	"Fair — add year, fuel type & engine size"
Make + Model + Year	70%	"Good — add fuel type & engine size to improve"
Make + Model + Year + Fuel + Engine CC	85%	"High — all details provided"

This is a UI prompt only — it never reaches the backend, and it disappears on submit, reappearing only when the form is cleared.

Type mode fuzzy matching in the backend:

1. `make_n = make.upper().strip()` — no fuzzy pass on the make initially; if the make yields no rows, a `fuzz.WRatio ≥ 75` make-level fuzzy pass runs.
2. `model_n = _norm_model_key(model)` — normalises separators and inserts letter/digit spaces.
3. `_fuzzy_model_score_for_year()` runs partial_ratio → token_set_ratio → WRatio with year-proximity tie-breaking, as described in Layer 1.
4. If nothing matches, alias resolution (Layer 2b) tries to map the typed name to a JDM equivalent.

Stop-start is an explicit input, not an inference. The form carries its own i-stop selector, defaulting to on, and its value is sent as `stop_start` on every request in both modes. `matcher._infer_stop_start()` — which guessed stop-start from make and year — still exists but is dead code, retained only in case that input is ever made optional again. It is documented here because earlier versions of this document described it as live behaviour; it is not.

8. Authentication & Security

8.1 Admin Authentication (`admin_auth.py`)

Admin accounts use **bcrypt** (cost 12) for password storage. Sessions are tracked in the `admin_sessions` table.

Login flow:

1. `POST /api/admin/login` with username + password (rate-limited 5/min).
2. If the account is locked, the request is rejected up front with the unlock time.
3. bcrypt comparison runs against a dummy hash even when the user does not exist, so a missing account and a wrong password take the same time.
4. On failure, `failed_login_attempts` increments; at **8 attempts** the account locks for **15 minutes**. A successful login resets both fields.
5. On success: generate a 64-char hex token (`secrets.token_hex(32)`), insert an `AdminSession` row expiring in 8 hours, and set an `HttpOnly; Secure; SameSite=Strict` cookie.

Per-request auth: every admin endpoint depends on `require_admin`, which reads the `admin_session` cookie, looks up the session row, checks expiry, deletes expired sessions as it finds them, and returns the `AdminUser`. All admin routes are covered except `/login` and `/logout`, which must be public.

Cookie security properties:

- `HttpOnly` — JavaScript cannot read the token (XSS mitigation).
- `SameSite=Strict` — blocks CSRF from cross-origin requests. CSRF tokens were explicitly declined on the strength of this.
- `Secure` — HTTPS only in production; disabled for local HTTP dev via `ADMIN_COOKIE_SECURE=false`. The same flag gates the HSTS response header.
- 8-hour TTL, matching the session row.

8.2 Admin Account Lifecycle

There is no CLI script for creating admins; `scripts/create_admin.py` was removed.

First admin (bootstrap). `GET /api/admin/bootstrap/status` and `POST /api/admin/bootstrap` back the public `/admin/setup` page. The endpoint is gated by two conditions: the `admin_users` table must be empty (checked first, so a populated table can never be used as a token-guessing oracle) *and* the request must present the `ADMIN_BOOTSTRAP_TOKEN` shared secret. Once any admin exists the endpoint permanently returns 403 regardless of token.

Bootstrap takes a `pg_advisory_xact_lock` before inserting. "Count the admins, then insert" is not atomic under Postgres's default READ COMMITTED isolation — concurrent transactions can all observe zero before any commits, and distinct usernames mean the unique constraint does not catch it. Eight concurrent requests produced three admins before the lock was added.

Every admin after the first is created from the dashboard via `POST /api/admin/users`, gated by `require_admin` — being logged in is the credential, so no token is involved.

Password reset — two independent mechanisms:

- An admin-generated link (`POST /api/admin/users/{id}/reset-link`): a logged-in admin generates a link for a locked-out colleague and passes it on by hand. Needs no email infrastructure and works today.
- Real email delivery via AWS SES (`email_utils.py`, `SES_FROM_EMAIL` / `SES_REGION`). SES needs domain verification performed in AWS outside this repository; until `SES_FROM_EMAIL` is set, `send_reset_email()` logs a warning and no-ops, so the link mechanism keeps working regardless.

Reset tokens are SHA-256 hashed rather than bcrypt — they are high-entropy CSPRNG values, so bcrypt's deliberate slowness buys nothing. They are single-use, overwritten on reissue, expire after **60 minutes**, and a successful reset invalidates every existing session for that admin.

Admin Users page (`/admin/users`) lists all admins and supports add, delete and reset-link generation. Two rules are enforced server-side: `PATCH /api/admin/users/{id}` returns 403 unless `id` is the caller's own account — letting one admin edit another's email would let them redirect that admin's password-reset destination — and delete blocks both self-deletion and deletion of the last remaining admin.

All of these events are written to `admin_account_audit_log` and surfaced as an Activity feed on the same page.

8.3 Test-User Authentication

Test users register via `POST /api/auth/register` and authenticate via `POST /api/auth/login`. Sessions use Bearer tokens held in client `localStorage` rather than cookies, with a 24-hour TTL. The token is sent as `Authorization: Bearer <token>` on feedback submissions so feedback can be attributed to an account.

8.4 Rate Limiting

All public endpoints are rate-limited via `slowapi`, keyed by client IP:

Endpoint	Limit
`/health`	60/min
`/api/recommend`	10/min
`/api/feedback`	5/min
`/api/makes`, `/api/models/{make}`, `/api/years`, `/api/batteries/search`	20/min
`/api/auth/register`	5/min

`/api/auth/login`	10/min
`/api/admin/login`	5/min

Exceeding a limit returns HTTP 429 with a `retry_after` field. Both `Limiter` instances (one in `app.py`, one in `admin_router.py`) share the single registered `RateLimitExceeded` handler.

Known limitation: `slowapi`'s default storage is in-process, so each application worker keeps its own counters and a per-IP limit is effectively multiplied by the number of workers behind the load balancer. Wiring both limiters to the already-provisioned Redis instance, and adding daily caps on top of the per-minute limits, is a planned change.

8.5 Least-Privilege Public Database Role

Public traffic and admin traffic no longer share a database role. `db.py` exposes two engines:

Engine	Role	Used by
`engine` / `SessionLocal`	`batteryfinder` (full privilege)	`admin_router.py`, `admin_auth.py`, pipelines, scripts
`public_engine` / `PublicSessionLocal`	`batteryfinder_public` (restricted)	`matcher.py`'s loaders, every public route in `app.py`, `app_state.py`

The restricted role is created by migration `20260721_c4d5e6f7a8b9` with exactly the grants it needs: `SELECT` on the matcher's seven read sources plus `app_settings` and `test_users`, and `INSERT` with a narrow `UPDATE` on `search_events`, `user_feedback`, `test_users`, `test_user_sessions` and `admin_stats`. Enforcement is now the database's job rather than a matter of application-code discipline.

Both pools are sized independently via environment variables (section 14.1), because the ceiling that matters is cluster-wide: every worker holds its own pools, so the real connection count is *workers × (DB_POOL_SIZE + DB_MAX_OVERFLOW + PUBLIC_DB_POOL_SIZE + PUBLIC_DB_MAX_OVERFLOW)*, which must stay under Postgres's `max_connections`. With the defaults that is 50 per worker, so a 4-worker deployment can demand 200 connections against a default limit of 100.

8.6 HTTP Security Headers

The `security_headers` middleware sets, on every response:

Header	Value
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`geolocation=(), camera=(), microphone=(), payment=(), usb=()`
`Content-Security-Policy`	see below
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains` (only when cookies are in secure mode)

The CSP was verified against the actual built frontend output: `default-src 'self'` with `script-src 'self'`, `object-src 'none'`, `base-uri 'self'`, `form-action 'self'` and `frame-ancestors 'none'`. Two deliberate relaxations exist — `style-src` allows `'unsafe-inline'` because the frontend renders inline `style={{}}` props throughout rather than utility classes, and `img-src` allows `cdn.jsdelivr.net` because the country-flag component fetches flag images from that CDN at runtime.

8.7 Path Handling

Two endpoints resolve a caller-supplied path, and both normalise and containment-check before trusting it:

- The SPA catch-all in `app.py` (section 3.4).
- `GET /api/admin/backup/{filename}/download`, which uses the same guard to keep downloads inside the backups directory.

One lower-severity instance remains open: `POST /api/admin/pipeline/crsp/validate` and one edit endpoint accept an arbitrary `file_path` in the request body and read it with `pandas`. Both are `require_admin`-gated and constrained to `.xlsx`/`.xls` suffixes, so the caller is already trusted, but they deserve the same normalise-and-contain treatment.

A broader sweep at the same time confirmed: SQL is built with bind parameters throughout (f-strings only splice in hardcoded table and column names); there is no `eval`, `exec`, `subprocess`, `os.system` or `pickle` anywhere in the codebase; all admin routes carry `require_admin` except login and logout; CORS uses an explicit allowlist with no wildcard; and FastAPI is not instantiated with `debug=True`, so unhandled exceptions return a generic 500 rather than a stack trace.

9. Admin Dashboard API

All routes are mounted under `/api/admin/` from `admin_router.py`. Every route requires a valid admin session cookie except `/login` and `/logout`. The surface is roughly 100 routes; they are grouped below.

9.1 Auth & Account Management

Endpoint	Description
`POST /login`, `POST /logout`, `GET /me`	Session lifecycle and current user
`GET /bootstrap/status`, `POST /bootstrap`	First-admin setup (see 8.2)
`POST /forgot-password`, `POST /reset-password`	Self-service reset
`GET /users`, `POST /users`	List and create admins
`PATCH /users/{id}`, `DELETE /users/{id}`	Self-only edit; delete with self/last-admin guards
`POST /users/{id}/reset-link`	Generate a manual reset link for another admin
`GET /users/audit-log`	Account-event history

9.2 Stats & Health

Endpoint	Description
`GET /stats`	Usage counters, per-table row counts (including `oem_fitment` and `vehicle_years_exide`), battery origin/technology breakdown, and today's match-type breakdown
`GET /health`	Connectivity, latency, and connection-pool usage

`GET /health` reports the **cluster-wide** connection count by querying `pg_stat_activity` and `pg_settings` directly, alongside this worker's own in-process pool figure. The per-process number alone was misleading, since each worker only ever knew its own share. Status is `ok`, `degraded` (latency > 500 ms or pool > 85% used), or `down`.

The public, unauthenticated `GET /health` at the application root is a different endpoint: it performs a real `SELECT 1` and returns only `{"status": "ok"}` or a 503, deliberately withholding latency and pool figures from

anonymous callers. It is also exempt from the maintenance-mode 503 override so a load balancer can still probe during maintenance.

9.3 Battery CRUD

Endpoint	Description
`GET /batteries`	Paginated list with search and technology filter
`POST /batteries`	Add a battery
`PATCH /batteries/{id}`	Update a battery
`DELETE /batteries/{id}`	Soft-delete a battery
`POST /batteries/upload`	Bulk upsert from Excel (10 MB cap, via `upload_pipeline.py`)
`GET /batteries/export`	Download all active batteries as `.xlsx`

Every mutation writes to `battery_audit_log` and clears and re-warms `matcher._batteries()` immediately, so the next recommendation sees the change.

9.4 Audit, Sessions & Settings

Endpoint	Description
`GET /audit`, `GET /audit/users`	Battery change history, filterable by action, admin, battery code and date range
`GET /sessions`, `DELETE /sessions/{id}`	Admin sessions, with revoke
`GET /test-sessions`, `DELETE /test-sessions/{id}`, `GET /test-users`	Test-user sessions and accounts
`GET /search-events`	Paginated search log
`GET /maintenance`, `POST /maintenance`	Read and toggle maintenance mode (persisted in `app_settings`)
`POST /cache/refresh`	Clear and re-warm every matcher cache, including `_fitment()`

9.5 Feedback Reports

Endpoint	Description
`GET /feedback`	Paginated feedback with accuracy filter
`GET /feedback/summary`	Aggregate statistics
`GET /feedback/export-pdf`	Full report as PDF (ReportLab)
`GET /feedback/export-csv`	Full report as CSV

9.6 Vehicle Data Browsing

Endpoint	Description
`GET /vehicle-data/core`	Paginated, searchable browse of `vehicle_years`
`GET /vehicle-data/oem-fitment`	Paginated, searchable browse of `oem_fitment`
`GET /vehicle-data/oem-fitment/by-make`	Per-make aggregate counts for the brand grid

9.7 CRSP Pipeline (vehicle year enrichment)

Endpoint	Description
`POST /pipeline/crsp/upload`, `POST /pipeline/crsp/validate`	Upload and pre-validate a raw CRSP Excel (10 MB cap)
`POST /pipeline/crsp/dataprep`	Run Stage 1
`POST /pipeline/crsp/run`	Run Stage 2 (year enrichment)

`GET /pipeline/crsp/status`, `POST .../pause`, `.../resume`, `.../cancel`	Long-run control
`GET /pipeline/crsp/staging`, `PATCH /pipeline/crsp/staging/{id}`	Review and edit staged rows
`GET /pipeline/crsp/staging/pre-commit`	Re-run conflict detection against current `vehicle_years`
`POST /pipeline/crsp/staging/commit`, `.../discard`	Commit or discard the run
`GET /pipeline/crsp/versions`, `.../{id}/changelog`, `POST .../{id}/revert`	Version history and rollback
`GET /pipeline/crsp/unmatched`, `POST /pipeline/crsp/rerun-unmatched`	Inspect and retry unresolved models
`GET /pipeline/crsp/provider-health`	Liveness probe across all external enrichment APIs
`POST /pipeline/crsp/fueleconomy/download`, `.../cancel`, `GET .../download-status`	Manage the fueleconomy.gov bulk CSV
`GET /pipeline/crsp/stats`, `.../cache-status`, `POST /pipeline/crsp/cache/refresh`	Dataset counts and cache age

9.8 Web Validation Pipeline

Endpoint	Description
`POST /pipeline/web-validate/run`, `GET .../status`, `POST .../pause`, `.../resume`, `.../cancel`	Run control
`GET /pipeline/web-validate/staging`, `PATCH .../staging/{id}`	Review and edit staged Ah/technology
`GET /pipeline/web-validate/staging/pre-commit`	Re-check conflicts against master
`POST /pipeline/web-validate/staging/commit`, `.../discard`	Commit or discard
`POST /pipeline/web-validate/staging/fix-ah`	Auto-correct suspicious Ah values via a fresh web lookup
`GET /pipeline/web-validate/versions`, `.../{id}/changelog`, `POST .../{id}/revert`	Version history and rollback
`PATCH /pipeline/web-validate/master/{id}`, `DELETE .../master/{id}`	Edit or delete a committed record
`GET /pipeline/web-validate/coverage`, `.../matched`, `.../providers`, `.../staleness`	Coverage reporting and provider status
`POST /pipeline/web-validate/reset-cache`	Clear the lookup cache file and matched rows
`GET /pipeline/quotas`	Per-provider quota usage (rolling monthly, or prepaid balance)

9.9 Coverage Expansion Pipeline

Endpoint	Description
`POST /pipeline/expansion/run`, `GET .../status`, `POST .../pause`, `.../resume`, `.../cancel`	Run control
`GET /pipeline/expansion/makes`, `POST`, `PATCH /{id}`, `DELETE /{id}`	Manage target makes and their per-source slugs

9.10 Backups

Endpoint	Description
`POST /backup`	Trigger an on-demand `pg_dump`

`GET /backup`	List available dumps
`GET /backup/log`	Backup history from `admin_backup_log`
`GET /backup/{filename}/download`	Download a dump (path-traversal guarded)

Backups keep the 10 most recent dumps and prune the rest. See section 14.3 for the two backup modes.

10. Frontend Architecture

10.1 Stack

Concern	Technology
Framework	React 19
Language	TypeScript (strict mode)
Build tool	Vite + Bun
Styling	Tailwind CSS v4 + custom CSS variables
Icons	Lucide React + Heroicons
UI primitives	Radix UI (label, progress, select, switch, tooltip)
HTTP client	Native `fetch` with typed wrappers in `lib/api.ts`
Routing	React Router 7 (SPA, served by the FastAPI catch-all)
Client-side PDF	jsPDF + jspdf-autotable (admin exports)
Testing	Vitest + Testing Library + jsdom

In development, Vite proxies `/api/*` to `localhost:8000`. In production, FastAPI serves the built `frontend/dist/` directory. All admin pages are loaded with `React.lazy()`, so the public bundle never carries dashboard code.

10.2 Key Components

`VehicleFinderForm.tsx` — the main public search form, and the desktop result card. All form state is local React state; there is no external state manager.

- `makes`, `models`, `years` — populated by cascading API calls in Select mode.
- `typeMode` — switches between Select and Type input modes.
- `istop` — i-stop (stop-start) toggle, defaults to `true`, sent as `stop_start` on every request in both modes.
- Estimated confidence bar (Type mode only) — computed from which fields are filled, shown before submission.
- On submit, calls `fetchRecommendation()` and renders the result panel inline.
- The result panel shows the primary battery (Ah, CCA, technology, battery code), the segmented Result Accuracy bar, the same-technology alternatives grid, an amber fallback warning when `matched_make` is null, and the "Rate this result" feedback trigger.

Note for maintainers: this file uses inline `style={{}}` props throughout rather than Tailwind utility classes. Extend it the same way — this is also why the CSP must allow `style-src 'unsafe-inline'`.

`MobileVehicleFinder.tsx` — a separate mobile-optimised finder with its own mode toggle and layout, sharing the same API layer and validation rules (including the 1960–2030 year bounds enforced client-side before submit).

`AccuracyBar.tsx` — four-segment coloured accuracy bar rendered below every recommendation. Each segment's width is proportional to its maximum contribution, and it fills based on match quality computed from `match_type`, `confidence`, `matched_year` and `matched_make`:

Segment	Colour	Max	What it measures
Car Found	Green `#10b981`	50	Whether the vehicle was found in the database (0 on rule-based fallback)
Year	Blue `#3b82f6`	25	Year delta from the matched row (25 exact, 20 delta-1, 13 delta-2, 8 delta-3)
Data Quality	Purple `#8b5cf6`	15	Real-world fitment data (15), structured dataset (12), engine estimate (4)
Engine Info	Amber `#f59e0b`	10	Whether engine details were confirmed from records or inferred

The total (0–100) is shown as a percentage alongside the bar, with a plain-language legend explaining each colour. Like the form, it uses inline styles.

`ConfidenceBar.tsx` — the simpler pre-submission confidence indicator used in Type mode.

`FeedbackModal.tsx` — collects 1–5 star satisfaction, an accuracy rating, which parts were wrong, and free-form comments; submits to `POST /api/feedback`, attaching the Bearer token when a test user is signed in.

Supporting components: `TermsGate` (first-visit terms acceptance), `PrivacyModal`, `CopyrightModal`, `FaqModal`, `HowToUseModal`, `BuildVersionModal`, `Notification`, `PageLoader`, `SkeletonForm`, and a small `ui/` set (`Badge`, `OrbitLoader`, `Skeleton`, `StartStopIcon`).

10.3 Admin Pages (`frontend/src/pages/admin/`)

Page	Route	Description
`AdminLogin.tsx`	`/admin/login`	Username/password form; sets the session cookie
`AdminSetup.tsx`	`/admin/setup`	First-admin bootstrap (public, token-gated)
`AdminForgotPassword.tsx`	`/admin/forgot-password`	Request a reset email
`AdminResetPassword.tsx`	`/admin/reset-password`	Consume a reset token
`AdminDashboard.tsx`	`/admin/dashboard`	Stats cards and the DB health card
`AdminBatteries.tsx`	`/admin/batteries`	Battery list with search, add, edit, delete, upload, export
`AdminAuditLog.tsx`	`/admin/audit`	Filterable battery audit log
`AdminFeedback.tsx`	`/admin/feedback`	Feedback table, summary stats, PDF/CSV export
`AdminSessions.tsx`	`/admin/sessions`	Admin sessions with revoke
`AdminTestSessions.tsx`	`/admin/test-sessions`	Test-user sessions and accounts
`AdminVehicleData.tsx`	`/admin/vehicle-data`	Browse `vehicle_years` and `oem_fitment`
`AdminDataPipeline.tsx`	`/admin/data-pipeline`	Run, monitor, review and commit every pipeline
`AdminSearchLog.tsx`	`/admin/search-log`	Search-event log

AdminUsers.tsx	/admin/users	Admin accounts and account-activity feed
AdminBackups.tsx	/admin/backups	Trigger, list and download database backups
AdminSettings.tsx	/admin/settings	Maintenance mode and other app settings

Every route above except login, setup, forgot-password and reset-password is wrapped in `ProtectedRoute`, inside the shared `AdminLayout` shell.

10.4 API Layer (`lib/api.ts`, `lib/adminApi.ts`, `lib/testUserApi.ts`)

All HTTP calls go through typed async wrappers. Errors are normalised: a `{ "detail": "..." }` body is re-thrown as an `Error` with that message, so components can catch everything with `err.message`.

```
export async function fetchRecommendation(payload: RecommendRequest):
Promise<RecommendationResult>
// Times out after 8 seconds — covers a cold backend still warming its caches
export async function fetchMakes(): Promise<string[]>
```

Supporting modules: `TestUserContext.tsx` (React context for the test-user session), `maintenance.ts` (detects the 503 maintenance response), `brandBranding.ts` / `makeBranding.ts` (per-brand presentation), and `utils.ts`.

10.5 Two Input Modes — UI Perspective

Select Mode (default):

- Three cascading `<select>` elements (Make → Model → Year).
- Models are disabled until a make is chosen; years until a model is chosen.
- Fuel type and engine CC are optional.
- A hint link — "Don't see your model? Type your query instead" — appears once a make is selected.

Type Mode:

- Three free-text `<input>` elements.
- Fuel type and engine CC are required, since the rule-based fallback needs them.
- The live estimated-confidence bar guides the user toward filling every field.
- The Type/Select toggle switches modes and resets all fields.

11. Data Pipelines

Pipelines live in `backend/pipelines/` and are run either from the admin Data Pipeline page (which spawns a background thread and streams progress) or directly from the CLI with `backend/` as the working directory. Long-running pipelines support pause, resume and cancel from the UI.

The important structural change since version 1.0: **no pipeline writes to a live table directly.** Enrichment output lands in a staging table, an admin reviews conflicts, and only then is it committed — producing a version row and a row-level changelog that makes the commit revertible.

11.1 The CRSP Pipeline (two stages)

```
Stage 1 — pipelines/dataprep.py
  Input:  a CRSP Excel file (uploaded via the admin UI, or a path on disk)
  Output: vehicles_raw table + battery_lookup_dataset.csv / .xlsx
  Does:   parses the raw CRSP dealer dataset, cleans and deduplicates by
          make/model/CC/fuel, and preserves the Model number column
          (the Japanese type-approval / chassis code) for Stage 2.

Stage 2 — pipelines/enrich_years.py
  Input:  battery_lookup_dataset.csv
  Output: vehicle_years_staging  (never vehicle_years directly)
  Does:   for each unique make/model, queries external sources for production
          year ranges, then writes staged rows with per-row validity and
          conflict flags for admin review.
```

Stage 2 source order (first usable answer wins):

#	Source	Notes
1	fueleconomy.gov bulk CSV	US EPA, 1984–present, exact per-year data. Cached in the `fueleconomy_vehicles` table
2	CarAPI	Community year ranges
3	CarQuery	Community year ranges
4	Japanese Wikipedia	Generation-level production data with chassis codes — `pipelines/ja_wikipedia_years.py`
5	English Wikipedia	Article extracts containing year mentions
6	NHTSA VPIC	Fallback model list; confirms a model exists but carries no per-year data

Wikidata was removed as a source. Its `P571` inception date has no lower bound and returns the *nameplate's* founding year rather than any generation's start — this is literally why a Land Cruiser was once stored as starting in 1951. Japanese Wikipedia supersedes it with real generation-level data.

`ja_wikipedia_years.py` is worth describing separately, since it solves a coverage problem no threshold change could. `ja.wikipedia.org` publishes a structured table per *generation* with exact production periods and manufacturer chassis codes, for Japanese and Western makes alike. The module resolves a title (English Wikipedia search → `langlinks` → the Japanese title, with a scored fallback and an honest "unresolved" rather than a guess), then extracts years from two shapes that must both be supported because articles often have one and not the other: section headings such as `40 系（1960 年 – 1984 年）`, and per-generation infobox 販売期間 fields. It parses Japanese date notation, imperial-era calendars, and open-ended ranges, and uses `mwparserfromhell` rather than regular expressions because real infobox values nest templates that a naive pattern silently mis-reads.

Two safety properties matter operationally. Requests are spaced at least 1 second apart, and any non-200 or non-JSON response raises a `ThrottledError` that propagates to every caller — a throttled response is **never** recorded as "no data", which is exactly how bad year data entered the table historically. And where a trim cannot be attributed to a specific generation unambiguously, the module returns no match with the candidates attached rather than guessing.

Chassis codes as matching input. CRSP's `Model number` column holds real Japanese type-approval codes (`3BA-VJA300W-GNUZZ`). `extract_chassis_code()` strips the classification prefix to recover the generation code (`VJA300W`), which disambiguates which generation a trim belongs to far more reliably than the model text. The code is carried through staging into `vehicle_years.model_number` but is deliberately excluded from conflict detection — it is a matching input, not a spec value worth blocking a review over.

`pipelines/oem_fitment_years.py` is a second, zero-cost year source that reads the `oem_fitment` table instead of any network service. It exists because Japanese Wikipedia documents JDM passenger cars well and commercial vehicles barely at all; for battery fitment specifically, the manufacturer's own guide is arguably the better authority anyway.

11.2 Web Battery Validation (`web_battery_validate.py`)

Queries external search providers in order — **SearXNG → DuckDuckGo → Brave → Tavily** — for the internet-accepted battery for each `(make, model, year)` not already present in `web_battery_validation`. Results go to `web_battery_staging` for review, then to the master table where `matcher._web_validation()` picks them up as Layer 0a.

Provider use is metered against `api_usage_quota`, with quota reservation done atomically so concurrent workers cannot both consume the last call. Quota bookkeeping is cached in memory and flushed once per run; before that, a single run could cost more than 10,000 database round-trips on quota checks alone. DuckDuckGo is unmetered and carries the bulk of the volume; Brave is a prepaid balance that never auto-resets, so it is queried at nameplate level rather than per variant.

The battery region for a resolved row always comes from the vehicle's own make, never from a "NS"/"DIN" keyword found in the scraped text — trusting the scraped value put DIN-coded batteries on 490 NS-region vehicles before it was fixed.

11.3 Other Pipelines

```
pipelines/exide_fitment_clean.py
  Input:  the Exide global fitment guide Excel
  Output: oem_fitment (full truncate + reload on every run)
  Note:   vehicle_years is never written to; vehicle_years_exide is a view
          over oem_fitment and therefore updates automatically.

pipelines/dial_battery_clean.py
  Input:  data/Dial-A-Battery Data - 2023-2026.xlsx
  Output: dial_battery_inference
  Does:   cleans and normalises call-centre order data, aggregates by
          vehicle type + model + battery code, counts frequency.

pipelines/vehicle_coverage_expansion.py
  Input:  the expansion_makes table (managed from the admin UI)
  Output: vehicle_years_staging (same review-and-commit workflow)
  Does:   expands coverage beyond the CRSP catalogue using NHTSA VPIC,
          CarQuery, cars-data.com and car.info, with a per-make year tier.

pipelines/generate_battery_lists.py   — battery list generation helper
pipelines/model_name_utils.py         — shared make/model cleaning helpers
```

`backend/scripts/` holds roughly fifty standalone remediation and reporting scripts (data fixes, quarantine passes, audits, workbook exports). They are not part of the application runtime. The convention they follow is worth knowing: dry-run by default with an explicit `--execute` flag, a real database backup taken first and logged to `admin_backup_log`, and a `vehicle_years_versions` / `vehicle_years_changelog` entry written so the change can be reverted from the admin UI like any other commit.

11.4 Upload Pipeline (`upload_pipeline.py`)

The admin dashboard's battery Upload feature uses this module. It accepts an Excel file up to 10 MB and performs:

- **Column aliasing** — accepts multiple column-name variants (`battery_code` / `code` / `sku`, `model_key` / `model`, `ah` / `capacity_ah`, …).

- **Row validation** — `battery_code`, `model_key`, `ah` (numeric, > 0) and `technology` are required; `cca_en` is optional. Any row missing a required field or failing a type check is rejected, and valid rows still commit.
- **Merge rules** — INSERT if new, UPDATE if changed, SKIP if identical.
- **Audit logging** — every INSERT and UPDATE writes to `battery_audit_log`.
- **Last-write-wins** — duplicate battery codes within one file take the last occurrence, with a warning.

Returns `{ inserted, updated, skipped_unchanged, rejected, errors }`.

12. Rate Limiting & Observability

12.1 Rate Limiting

Implemented with `slowapi`, keyed by client IP; per-route limits are listed in section 8.4. On `RateLimitExceeded` the API returns:

```
{
  "detail": "Rate limit exceeded. Please try again later.",
  "retry_after": "10 second"
}
```

Registering that handler was itself a fix — the function existed but was never wired up, so exceeding a limit previously produced an unhandled 500 instead of a clean 429.

12.2 Read Counter

The admin dashboard's "reads today" figure counts `POST /api/recommend` calls:

1. `count_reads_middleware` increments an in-memory counter on each recommendation request.
2. `_flush_read_counter()` fires every 60 seconds and writes the accumulated delta to `admin_stats` (`stat_key = "db_reads"`) in one write.
3. `_reset_daily_reads()` fires at local midnight, zeroing both the stored value and any pending in-memory delta.

12.3 Search Events

Every recommendation also writes a `search_events` row from a background thread: what was asked for, what was returned, the match type and confidence, whether anything matched, a truncated client IP, and whether the request came from Select or Type mode. This is the data behind the admin Search Log and the dashboard's match-type breakdown, and it is the primary signal for deciding which vehicles deserve manual data attention.

The write is one INSERT per request on the public role's connection. At current volumes that is fine; it is the first thing to batch if traffic grows substantially, and `search_events` retention is the main driver of long-term database size.

12.4 Health Endpoints

`GET /api/admin/health` (admin-only) returns full diagnostics:

```
{
  "status":              "ok",
  "connected":           true,
  "latency_ms":          3.2,
  "cluster_connections": 14,
  "max_connections":     100,
  "pool_size":           10,
```

```
  "pool_checked_out":    1,
  "pool_usage_pct":      9.1
}
```

`cluster_connections` comes from `pg_stat_activity` and `pg_settings` — ground truth from Postgres itself, independent of which worker answers the request. The per-worker pool figures are reported alongside it as a secondary "this instance" reading. Status degrades to `degraded` above 500 ms latency or 85% pool usage, and `down` when the database is unreachable.

`GET /health` (public, unauthenticated, 60/min) performs a real `SELECT 1` and returns only `{"status": "ok"}` or a 503 — intended for the load balancer's target-group health check, and deliberately exempt from the maintenance-mode override so probes still succeed during maintenance.

13. Automated Test Suite

13.1 Backend (`backend/tests/`, pytest)

`conftest.py` wires a minimal FastAPI app around `admin_router`'s routes against a real local Postgres, and wraps each test in a transaction that is rolled back afterwards, so database writes never persist. A separate `public_db_session` fixture keeps tests that exercise public-route writes inside that same transaction rather than leaking rows through the restricted role's own connection.

One caveat worth knowing: rollback covers database state only, **not filesystem side effects**. A route that deletes a cache file on disk needs that call mocked directly.

File	Covers
`test_matcher.py`	Normalisation, Ah brackets, technology rules, each matching layer
`test_admin_battery_crud.py`	Battery CRUD edge cases, including soft-deleted code reuse
`test_admin_crsp_pipeline.py`	CRSP staging commit / discard / revert lifecycle
`test_admin_web_validate_pipeline.py`	Web-validation staging and versioning
`test_web_validate_staging_and_quota.py`	Staging lifecycle and the quota race condition
`test_web_validate_region_resolution.py`	DIN/NS region resolution, both directions
`test_admin_account_management.py`	Bootstrap, reset, user management, bootstrap concurrency
`test_admin_backup.py`	Backup success/failure logging, listing, traversal rejection
`test_admin_vehicle_data.py`	Vehicle-data browse endpoints
`test_admin_health.py`, `test_health_endpoint.py`	Both health endpoints
`test_enrich_years_matching.py`	Fuzzy-match case handling, trim stripping, span capping, chassis-code parsing
`test_ja_wikipedia_parser.py`	Heading and infobox extraction, trim-to-generation matching

Several of these tests were written because a bug was suspected, and found real ones: a 500 on reusing a soft-deleted battery code, a model/schema type mismatch that made two staging filters fail in production, and the bootstrap race described in 8.2.

```
cd backend
pytest                              # run everything
pytest --junitxml=test-results.xml  # what CI runs
```

13.2 Frontend (`frontend/`, Vitest)

Component and module tests live beside their subjects as `*.test.tsx` / `*.test.ts`, covering `VehicleFinderForm`, `AccuracyBar`, `ConfidenceBar`, the admin pages, `ProtectedRoute`, `AdminLayout`, and the API and utility modules.

```
cd frontend
bun run test        # watch-free run
bun run test:ci     # JUnit report for Jenkins
```

14. Deployment & Environment

14.1 Environment Variables

`.env` lives at the repository root and is loaded by `python-dotenv` in `db.py`, which walks up from wherever the process started. It is loaded with `override=False`, so real environment variables — the ones ECS injects from SSM Parameter Store and Secrets Manager in production — always win over anything in the file. A stray `.env` baked into a production image therefore cannot clobber a real secret.

Variable	Required	Purpose
`DATABASE_URL`	Yes	Connection string for the full-privilege `batteryfinder` role
`PUBLIC_DATABASE_URL`	Yes	Connection string for the restricted `batteryfinder_public` role. Same host and database, different user
`PUBLIC_DB_PASSWORD`	Migration only	Password used when migration `20260721_c4d5e6f7a8b9` creates the restricted role. Not read by the running app
`ADMIN_COOKIE_SECURE`	No	`false` for local HTTP dev (default `true`); also gates the HSTS header
`ADMIN_BOOTSTRAP_TOKEN`	Before first setup	Shared secret required by `/admin/setup`. Irrelevant once any admin exists
`ALLOWED_ORIGINS`	No	Comma-separated CORS allowlist. Empty by default — the SPA and API are same-origin
`DB_POOL_SIZE` / `DB_MAX_OVERFLOW`	No	Admin pool sizing (defaults 10 / 15)
`PUBLIC_DB_POOL_SIZE` / `PUBLIC_DB_MAX_OVERFLOW`	No	Public pool sizing; each falls back to the corresponding admin value
`DB_BACKUP_MODE`	No	`docker` (default) or `direct` — see 14.3
`DB_BACKUP_CONTAINER` / `DB_BACKUP_USER` / `DB_BACKUP_NAME`	No	Backup target overrides
`SES_FROM_EMAIL` / `SES_REGION`	No	Verified SES sender for reset emails; the feature no-ops with a warning if unset

SEARXNG_URL	No	SearXNG base URL for web validation; skipped if unset
BRAVE_SEARCH_API_KEY / BRAVE_MONTHLY_QUOTA	No	Brave provider and its quota override (default 2000)
TAVILY_API_KEY / TAVILY_MONTHLY_QUOTA	No	Tavily provider and its quota override (default 1000)

Pool-size variables are parsed by a helper that **raises on a malformed or negative value rather than falling back silently** — a typo in a pool-size variable would otherwise stay invisible until the application ran out of connections under load. Maintenance mode is not an environment variable; see section 3.4.

14.2 Dependency Sets

`requirements.txt` is split three ways, each including the previous:

File	Contents
requirements.txt	Runtime only — everything needed to serve the app. This is all a production web host needs
requirements-pipeline.txt	Adds pipeline dependencies (BeautifulSoup, lxml, `mwparserfromhell`, DuckDuckGo and Tavily clients)
requirements-dev.txt	Adds pytest, httpx, type stubs and the notebook tooling. This is what CI installs

Two allocations are easy to get wrong and are worth stating explicitly: `xlrd` is **runtime**, not pipeline, because the admin upload endpoints accept `.xls` and pandas needs it; and `lxml` is pipeline-only despite never being imported by name, because it is passed as a string to BeautifulSoup.

On Windows with recent Python versions, `mwparserfromhell` has no prebuilt wheel and would need MSVC build tools. Install it with `WITH_EXTENSION=0` to use its documented pure-Python tokenizer instead — same library, same behaviour, no compiler.

14.3 Database Backups

`db_backup.py` supports two modes, chosen with `DB_BACKUP_MODE`:

- **`docker`** (default) — runs `pg_dump` inside the local Postgres container via `docker exec`, matching what `db-backup-script/backup-db.ps1` does.
- **`direct`** — runs a local `pg_dump` over TCP, with host, port, user, database and password all derived from `DATABASE_URL` itself, so a backup can never target a different database than the one being served. This is the mode that works against RDS and against a native Postgres install on a self-hosted box. The password is passed through `PGPASSWORD` in the child environment, never in the argument list, because the process table is world-readable.

Backups are written to `backups/`, the ten most recent are kept, and every attempt — success or failure — is recorded in `admin_backup_log`. A failed direct dump removes its own zero-byte output file, which `pg_dump` creates before it has even connected and which would otherwise be listed as a real backup.

14.4 Starting the Server

Development:

```
docker compose up -d              # local Postgres 15 on host port 5434
cd backend
```

```
pip install -r requirements-dev.txt
alembic upgrade head
uvicorn app:app --reload          # http://localhost:8000

cd frontend && bun install && bun run dev    # http://localhost:5173
```

Production (behind a load balancer):

```
cd frontend && bun run build      # outputs to frontend/dist/
uvicorn app:app --host 0.0.0.0 --port 8000 \
        --proxy-headers --forwarded-allow-ips="*"
```

FastAPI detects `frontend/dist/` at startup and serves it; all non-API paths fall through to `index.html`.

> ***--forwarded-allow-ips="*" is safe only when the application is unreachable except through the load balancer**, as on Fargate. On a self-hosted single VM where nginx proxies to uvicorn on the same host, uvicorn is directly reachable, so `"*"` lets any caller that reaches it spoof `X-Forwarded-For` and defeat every per-IP rate limit. Use `--forwarded-allow-ips="127.0.0.1"` and bind to `127.0.0.1` instead of `0.0.0.0` there.*

Note that the local Postgres container publishes on host port **5434**, not 5432. A native Postgres install commonly already owns 5432, and on Windows Docker Desktop would bind the container's side on IPv6 rather than erroring — silently splitting `localhost:5432` across two different databases.

14.5 CI/CD (Jenkinsfile)

#	Stage	What it does
1	Checkout	Pull source
2	Set up Python	`pip install -r backend/requirements-dev.txt`
3	Set up Node.js & Bun	Install Bun, then `bun install`
4	Lint Frontend	`bun run lint`
5	Build Frontend	`bun run build`
6	Test Frontend	`bun run test:ci` (Vitest, JUnit output)
7	Start test database	`docker compose up -d` then `alembic upgrade head`
8	Test Python	`pytest --junitxml=test-results.xml`
9	Archive Artifacts	Store `frontend/dist/**`

The post block always publishes JUnit results and tears the database container down. The Python test stage genuinely fails the build — an earlier version swallowed pytest failures silently.

14.6 First-Time Setup

```
# 1. Environment
#    Create .env at the repo root with DATABASE_URL and PUBLIC_DATABASE_URL
#    (see 14.1, and README.md's Environment Variables section)

# 2. Database
docker compose up -d
cd backend
alembic upgrade head          # also creates the batteryfinder_public role
                              # when PUBLIC_DB_PASSWORD is set
```

```
# 3. First admin account
#    Set ADMIN_BOOTSTRAP_TOKEN in .env, start the app, then visit /admin/setup.
#    There is no CLI script — see section 8.2.

# 4. Seed vehicle and battery data
python pipelines/dataprep.py
python pipelines/enrich_years.py        # → staging; review and commit in the UI
python pipelines/dial_battery_clean.py
python pipelines/exide_fitment_clean.py
#    Seed battery_specs separately via the admin Batteries upload.

# 5. Start the server
uvicorn app:app --reload
```

End of document.